Database Users, Roles, Privileges, and Access Control

Group 6 (HabalGo - Ride and Booking Management System)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** HabalGo - Ride and Booking Management System **Repository Link: https://github.com/jeanaroyo/HabalGo.git** **Our Group Repository:**
Projects set up	**The HabalGo repository was used as the basis for implementing the database access-control plan. The project is a motorcycle ride-hailing application designed to connect passengers with motorcycle drivers and manage related information, including user accounts, ride bookings, and other transactions.** **The first step was to examine the existing project files and understand how the application connects to its database. The database configuration and existing SQL files were reviewed to identify the database name, tables, and other database objects that needed access restrictions.** **The database used for this activity was named [INSERT ACTUAL DATABASE NAME]. The existing database structure was retained, and the access-control implementation was performed separately to avoid changing the original system functionality.** **The existing database tables should be identified from the actual HabalGo database structure before documenting their names and purposes.** **After reviewing the database structure, a separate folder named Activity-4 was prepared to organize the SQL scripts and access-control documentation.** **The following files were created:** 1. **01_create_roles.sql** 2. **02_grant_privileges.sql** 3. **03_revoke_privileges.sql** 4. **04_test_privileges.sql**

	5. **access_control_plan.md** **These files were organized to make the implementation easier to understand, execute, and maintain.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each role. The implementation follows the Principle of Least Privilege, meaning that each role should only receive the permissions necessary for its responsibilities.** **Three PostgreSQL group roles were established.** **Administrator Role (habalho_admin_role)** **The administrator role is responsible for managing and maintaining the HabalGo database. It receives broad privileges on the existing tables and sequences within the public schema.** **Its responsibilities include viewing database records, inserting new records, updating existing information, and deleting records when necessary.** **Application Role (habalho_app_role)** **The application role is intended for the HabalGo application that interacts with the database. It allows the retrieval, addition, and updating of database records.** **Its responsibilities include accessing information needed by the application, adding new records, and updating existing records.** **The application role is restricted from deleting records and creating database objects to help prevent unauthorized data removal and structural changes.** **Auditor Role (habalho_auditor_role)** **The auditor role is intended for users who only need to view database information. It receives SELECT privileges but cannot insert, update, or delete records.** **Its purpose is to allow authorized users to inspect HabalGo database information without making unauthorized changes.**

Access-Control Matrix

Database Operations	Administrator	Application	Auditor
SELECT	Allowed	Allowed	Allowed
INSERT	Allowed	Allowed	Denied
UPDATE	Allowed	Allowed	Denied
DELETE	Allowed	Denied	Denied
CREATE	Allowed	Denied	Denied

Database Users and Role Assignments

The following database login accounts were prepared:

Database User	Assigned Role
habalgo_admin_user	habalgo_admin_role
habalgo_app_user	habalgo_app_role
habalgo_auditor_	habalgo_auditor_

	user role **The three group roles were created with the NOLOGIN attribute. This means they function as permission groups and must be assigned to actual login accounts before individual users or application accounts can use their permissions.**
SQL scripts summary	**Four main SQL implementation and verification scripts were prepared to implement and check the access-control configuration.** **File 1: 01_create_roles.sql** **The first SQL script was used to create the three PostgreSQL group roles and their corresponding login accounts.** **The following group roles were created:** ● **habalgo_admin_role** ● **habalgo_app_role** ● **habalgo_auditor_role** **The following database login accounts were also created:** ● **habalgo_admin_user** ● **habalgo_app_user** ● **habalgo_auditor_user** **Each login account was assigned to its corresponding group role using PostgreSQL GRANT statements.** **This allows the database users to inherit the permissions assigned to their respective roles.** **File 2: 02_grant_privileges.sql** **The second SQL script was used to assign the required database permissions to each role.** **First, the three roles were given CONNECT permission on the current database and USAGE permission on the public schema.**

	The administrator role received ALL PRIVILEGES on all existing tables and sequences in the public schema. It was also given CREATE permission on the public schema. **The application role received SELECT, INSERT, and UPDATE permissions on all existing tables in the public schema. It also received the necessary sequence permissions for database operations.** **The auditor role received SELECT privileges only on all existing tables in the public schema.** **The use of ON ALL TABLES IN SCHEMA public allows the permissions to be assigned without manually specifying each existing table name.** **Default privileges were also configured where applicable so that future database objects can receive the intended permissions when created by the appropriate object owner.** **File 3: 03_revoke_privileges.sql** **The third SQL script was used to remove privileges that should not be available to restricted roles.** **The application role was restricted from performing DELETE operations.** **The auditor role was restricted from performing INSERT, UPDATE, DELETE, and other unnecessary modification operations.** **The script also ensures that the application role retains SELECT, INSERT, and UPDATE permissions, while the auditor role retains SELECT permissions.** **The application and auditor roles were also restricted from creating objects in the public schema.** **This helps maintain the Principle of Least Privilege by removing unnecessary access rights.** **File 4: 04_test_privileges.sql** **The fourth SQL script was used to check whether the assigned**

	permissions were correct. **The script contains queries that display the created roles, list their table privileges, and test specific permissions using PostgreSQL's has_table_privilege() function.** **The verification focused on checking whether:** ● **The administrator can perform SELECT operations.** ● **The administrator can perform INSERT operations.** ● **The administrator can perform UPDATE operations.** ● **The administrator can perform DELETE operations.** ● **The application role can view records.** ● **The application role can insert records.** ● **The application role can update records.** ● **The application role cannot delete records.** ● **The auditor role can select records.** ● **The auditor role cannot insert records.** ● **The auditor role cannot update or delete records.** **These checks help confirm that each role follows the intended access-control plan.**
Verification results	**After preparing the SQL scripts, they were executed in the PostgreSQL Query Tool through pgAdmin 4.** **The scripts were executed in the correct order to avoid permission-related errors.** **First Verification: Role Creation** **The first script was executed to create the three group roles and their corresponding login accounts.** **A verification query was used to display the created roles from the PostgreSQL system catalog.** **Expected result: All three group roles and their login accounts should appear in the query results.** **Second Verification: Privilege Assignment** **The second script was executed to assign the required database,**

schema, table, and sequence privileges.

The execution messages were checked to determine whether the commands completed successfully.

Expected result: Each role should receive the permissions defined in the access-control plan.

Third Verification: Privilege Restrictions

The third script was executed to remove unnecessary privileges from restricted roles.

Expected result:

- **The application role should not have DELETE privileges.**
- **The auditor role should not have INSERT, UPDATE, or DELETE privileges.**
- **The application role should retain SELECT, INSERT, and UPDATE privileges.**
- **The auditor role should retain SELECT privileges.**

Fourth Verification: Permission Checking

The fourth script was executed to verify the permissions using PostgreSQL queries.

The expected results were:

Verification	Expected Result
Administrator can SELECT records	TRUE
Administrator can INSERT records	TRUE
Administrator can UPDATE	TRUE

	records **Administrator can DELETE records** **TRUE** **Application can SELECT records** **TRUE** **Application can INSERT records** **TRUE** **Application can UPDATE records** **TRUE** **Application can DELETE records** **FALSE** **Auditor can SELECT records** **TRUE** **Auditor can INSERT records** **FALSE** **Auditor can UPDATE records** **FALSE** **Auditor can DELETE records** **FALSE** **The expected results demonstrate that the administrator has the intended database management permissions, the application role can perform normal application operations, and the auditor role is restricted to viewing records.** **Note: These are expected results, not confirmed execution results.**

	Actual results must be recorded after running the scripts and testing the permissions using the assigned roles. --- ## Screenshots **The following screenshots should be included as supporting evidence:** ● **Screenshot 1: G6_create_roles.png** ● **Screenshot 2: G6_grant_privileges.png** ● **Screenshot 3: G6_revoke_privileges.png** ● **Screenshot 4: G6_test_privileges.png** **The screenshots serve as supporting evidence of the SQL execution and verification process.**
Issues and fixes	**During the preparation and implementation of the access-control configuration, the following possible issues were considered and addressed.** ### Database Name Mismatch **The SQL scripts must be executed inside the correct database. The actual HabalGo database name must be used consistently to avoid applying the configuration to the wrong database.** ### Existing Database Roles **Running role creation commands repeatedly may cause errors if the roles already exist. The first SQL script checks whether each role exists before creating it.** ### Incorrect Execution Order **Granting permissions before creating the roles can result in errors. To prevent this, the scripts were executed in order, starting with role creation, followed by privilege assignment, privilege restrictions, and verification.** ### Insufficient Database Permissions

	Some PostgreSQL commands require appropriate administrative privileges. The scripts should be executed using an authorized PostgreSQL administrator account. **Incorrect Table Names** **Granting privileges to tables that do not exist can result in relation errors.** **To avoid this issue, the privilege assignment script uses:** **ON ALL TABLES IN SCHEMA public** **This allows PostgreSQL to apply the specified privileges to all existing tables in the public schema without manually entering individual table names.** **Future Database Objects** **Newly created tables and sequences may not automatically receive the same permissions as existing objects. Default privileges should be configured for the correct object owner when necessary.** **Application and Auditor Permission Restrictions** **The application role should not be able to delete records, while the auditor role should not be able to modify database records.** **The verification script checks that the application role can SELECT, INSERT, and UPDATE while DELETE is denied. It also checks that the auditor role can SELECT while INSERT, UPDATE, and DELETE are denied.**
Feedback to owners	**The HabalGo motorcycle ride-hailing application provides database functionality for managing information related to passengers, drivers, and ride transactions. However, implementing Role-Based Access Control can further improve the security and organization of database operations.** **The following recommendations are provided to the project owners:** ● **Maintain separate database roles for administrators, application access, and auditors.** ● **Avoid using administrator-level database credentials for regular**

	application activities. ● Assign actual login accounts to the appropriate group roles. ● Use strong passwords and protect database credentials. ● Review database permissions whenever new tables or features are added. ● Restrict access to sensitive passenger and driver information. ● Regularly check database roles and privileges to prevent unnecessary access. ● Use read-only accounts for users who only need to view database information. ● Perform regular database backups to help prevent data loss. These recommendations can help improve the security and maintainability of the HabalGo application.
AI usage declaration	Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation. AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, and identify possible implementation issues and their corresponding solutions. The SQL scripts should be reviewed and executed using PostgreSQL through pgAdmin 4. The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.